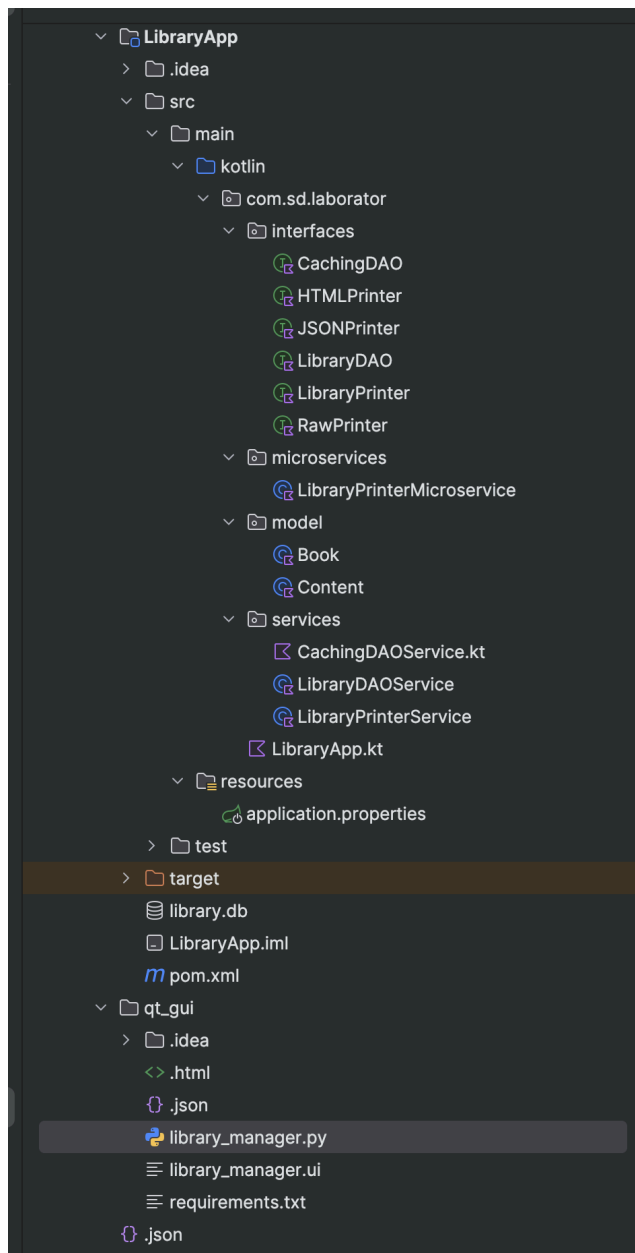


Pornind de la exemplul 2 din laboratorul 6, să se modifice acesta astfel încât să utilizeze un fișier pe disc pe post de memorie cache (se va utiliza numai adăugare și analiza pe bază de marcate temporale).

Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se va utiliza orchestrarea și respectarea principiilor SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.



Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.sd.laborator</groupId>
  <artifactId>LibraryApp</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>com.sd.laborator LibraryApp</name>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>2.1.9.RELEASE</version>
    <relativePath/> <!-- lookup parent from repository -->
  </parent>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <kotlin.version>1.3.61</kotlin.version>
    <kotlin.code.style>official</kotlin.code.style>
    <junit.version>4.12</junit.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-stdlib</artifactId>
      <version>${kotlin.version}</version>
    </dependency>
    <dependency>
      <groupId>org.jetbrains.kotlin</groupId>
      <artifactId>kotlin-test-junit</artifactId>
      <version>${kotlin.version}</version>
      <scope>test</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-amqp</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```

        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-jdbc</artifactId>
    </dependency>
    <dependency>
        <groupId>org.xerial</groupId>
        <artifactId>sqlite-jdbc</artifactId>
        <version>3.28.0</version>
    </dependency>
</dependencies>

<build>
    <sourceDirectory>src/main/kotlin</sourceDirectory>
    <testSourceDirectory>src/test/kotlin</testSourceDirectory>

    <plugins>
        <plugin>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-plugin</artifactId>
            <version>${kotlin.version}</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <phase>compile</phase>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                    <configuration>
                        <compilerPlugins>
                            <plugin>spring</plugin>
                        </compilerPlugins>
                    </configuration>
                </execution>
                <execution>
                    <id>test-compile</id>
                    <phase>test-compile</phase>
                    <goals>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
    <dependencies>
        <dependency>
            <groupId>org.jetbrains.kotlin</groupId>
            <artifactId>kotlin-maven-allopen</artifactId>
            <version>${kotlin.version}</version>
        </dependency>
    </dependencies>
</build>

```

```

        </dependencies>
    </plugin>

    <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
</plugins>
</build>

</project>

```

CatchingDAO

```

package com.sd.laborator.interfaces

interface CatchingDAO {
    fun exists(query: String): String
    fun addToCache(query: String, result: String)
}

```

HTMLPrinter

```

package com.sd.laborator.interfaces

import com.sd.laborator.model.Book

interface HTMLPrinter {
    fun printHTML(books: Set<Book>): String
}

```

JSON Printer

```

package com.sd.laborator.interfaces

import com.sd.laborator.model.Book

interface JSONPrinter {
    fun printJSON(books: Set<Book>): String
}

```

LibraryDAO

```

package com.sd.laborator.interfaces

import com.sd.laborator.model.Book

```

```
interface LibraryDAO {
    fun getBooks(): Set<Book>
    fun addBook(book: Book)
    fun findAllByAuthor(author: String): Set<Book>
    fun findAllByTitle(title: String): Set<Book>
    fun findAllByPublisher(publisher: String): Set<Book>
}
```

LibraryPrinter

```
package com.sd.laborator.interfaces

interface LibraryPrinter: HTMLPrinter, JSONPrinter, RawPrinter
```

RawPrinter

```
package com.sd.laborator.interfaces

import com.sd.laborator.model.Book

interface RawPrinter {
    fun printRaw(books: Set<Book>): String
}
```

LibraryPrinterMicroservice

```
package com.sd.laborator.microservices

import com.sd.laborator.interfaces.CachingDAO
import com.sd.laborator.interfaces.LibraryDAO
import com.sd.laborator.interfaces.LibraryPrinter
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.stereotype.Controller
import org.springframework.web.bind.annotation.RequestMapping
import org.springframework.web.bind.annotation.RequestMethod
import org.springframework.web.bind.annotation.RequestParam
import org.springframework.web.bind.annotation.ResponseBody

@Controller
class LibraryPrinterMicroservice {
    @Autowired
    private lateinit var libraryDAO: LibraryDAO

    @Autowired
    private lateinit var libraryPrinter: LibraryPrinter

    @Autowired
```

```

private lateinit var cachingDAO: CachingDAO

@RequestMapping("/print", method = [RequestMethod.GET])
@ResponseBody
fun customPrint(@RequestParam(required = true, name = "format", defaultValue = "") format: String): String {

    return when(format) {
        "html" -> {
            if(cachingDAO.exists("/print/html") != libraryDAO.getBooks().toString())
                cachingDAO.addToCache("/print/html", libraryDAO.getBooks().toString())
            return libraryPrinter.printHTML(libraryDAO.getBooks())
        }
        "json" -> {
            if(cachingDAO.exists("/print/json") != libraryDAO.getBooks().toString())
                cachingDAO.addToCache("/print/json", libraryDAO.getBooks().toString())
            return libraryPrinter.printJSON(libraryDAO.getBooks())
        }
        "raw" -> {
            if(cachingDAO.exists("/print/raw") != libraryDAO.getBooks().toString())
                cachingDAO.addToCache("/print/raw", libraryDAO.getBooks().toString())
            return libraryPrinter.printRaw(libraryDAO.getBooks())
        }
        else -> {
            if(cachingDAO.exists("/print/*****") != "Not implemented")
                cachingDAO.addToCache("/print/*****", "Not implemented")
            return "Not implemented"
        }
    }
}

@RequestMapping("/find", method = [RequestMethod.GET])
@ResponseBody
fun customFind(@RequestParam(required = false, name = "author", defaultValue = "") author: String,
               @RequestParam(required = false, name = "title", defaultValue = "") title: String,
               @RequestParam(required = false, name = "publisher", defaultValue = "") publisher: String): String {
    if (author != "") {
        if(cachingDAO.exists("/find/$author") != this.libraryDAO.findAllByAuthor(author).toString())

```

```

        cachingDAO.addToCache("/find/$author",
this.libraryDAO.findAllByAuthor(author).toString())
        return
this.libraryPrinter.printJSON(this.libraryDAO.findAllByAuthor(author))
    }
    if (title != "") {
        if(cachingDAO.exists("/find/$title") !=
this.libraryDAO.findAllByTitle(title).toString())
            cachingDAO.addToCache("/find/$title",
this.libraryDAO.findAllByTitle(title).toString())
            return
this.libraryPrinter.printJSON(this.libraryDAO.findAllByTitle(title))
    }
    if (publisher != "") {
        if(cachingDAO.exists("/find/$publisher") !=
this.libraryDAO.findAllByPublisher(publisher).toString())
            cachingDAO.addToCache("/find/$publisher",
this.libraryDAO.findAllByPublisher(publisher).toString())
            return
this.libraryPrinter.printJSON(this.libraryDAO.findAllByPublisher(publisher))
    }

    if(cachingDAO.exists("/find/*****") != "Not a valid field")
        cachingDAO.addToCache("/find/*****", "Not a valid field")
    return "Not a valid field"
}
}

```

Book

```

package com.sd.laborator.model

class Book(private var data: Content) {

    var name: String?
    get() {
        return data.name
    }
    set(value) {
        data.name = value
    }

    var author: String?
    get() {
        return data.author
    }
    set(value) {
        data.author = value
    }
}

```

```

    }

    var publisher: String?
        get() {
            return data.publisher
        }
        set(value) {
            data.publisher = value
        }

    var content: String?
        get() {
            return data.text
        }
        set(value) {
            data.text = value
        }

    fun hasAuthor(author: String): Boolean {
        return data.author.equals(author)
    }

    fun hasTitle(title: String): Boolean {
        return data.name.equals(title)
    }

    fun publishedBy(publisher: String): Boolean {
        return data.publisher.equals(publisher)
    }
}

```

Content

```

package com.sd.laborator.model

data class Content(var author: String?, var text: String?, var name: String?,
var publisher: String?)

```

CatchingDAOService

```

package com.sd.laborator.services

import com.sd.laborator.interfaces.CachingDAO
import org.springframework.beans.factory.annotation.Autowired
import org.springframework.jdbc.core.JdbcTemplate

```



```

import org.springframework.jdbc.core.RowMapper
import org.springframework.stereotype.Service
import java.sql.ResultSet
import java.sql.SQLException

class QueryResultRowMapper : RowMapper<String> {
    @Throws(SQLException::class)
    override fun mapRow(rs: ResultSet, rowNum: Int): String {
        return rs.getString("result")
    }
}

@Service
class CachingDAOService: CachingDAO{

    @Autowired
    private lateinit var jdbcTemplate: JdbcTemplate

    private fun createTableIfNotExists() {
        jdbcTemplate.execute("""CREATE TABLE IF NOT EXISTS cache(
                                id INTEGER PRIMARY KEY AUTOINCREMENT,
                                timestamp BIGINT,
                                query VARCHAR(100),
                                result VARCHAR(1024)""")
    }

    override fun exists(query: String): String {
        createTableIfNotExists()
        var max_age = System.currentTimeMillis() - 1000*60*60 // One hour
        var result: MutableList<String> = jdbcTemplate.query("SELECT result FROM
cache WHERE query='$query' AND timestamp >= $max_age", QueryResultRowMapper())
        if(result.size > 0){
            return result.first()
        }
        return ""
    }

    override fun addToCache(query: String, result: String) {
        createTableIfNotExists()
        jdbcTemplate.update("INSERT INTO cache(timestamp, query, result) VALUES
(?, ?, ?)", System.currentTimeMillis(), query, result)
    }
}

```

LibraryDAOService

```
package com.sd.laborator.services
```

```

import com.sd.laborator.interfaces.LibraryDAO
import com.sd.laborator.model.Book
import com.sd.laborator.model.Content
import org.springframework.stereotype.Service

@Service
class LibraryDAOService: LibraryDAO {
    private var books: MutableSet<Book> = mutableSetOf(
        Book(Content("Roberto Ierusalimschy","Preface. When Waldemar, Luiz, and
I started the development of Lua, back in 1993, we could hardly imagine that it
would spread as it did. ...","Programming in LUA","Teora")),
        Book(Content("Jules Verne","Nemaipomeniti sunt francezii astia! -
Vorbiti, domnule, va ascult! ....","Steaua Sudului","Corint")),
        Book(Content("Jules Verne","Cuvant Inainte. Imaginatia copiilor - zicea
un mare poet romantic spaniol - este asemenea unui cal nazdravan, iar
curiozitatea lor e pintenul ce-l fugareste prin lumea celor mai indraznete
proiecte.","O calatorie spre centrul pamantului","Polirom")),
        Book(Content("Jules Verne","Partea intai. Naufragiati vazduhului.
Capitolul 1. Uraganul din 1865. ...","Insula Misterioasa","Teora")),
        Book(Content("Jules Verne","Capitolul I. S-a pus un premiu pe capul unui
om. Se ofera premiu de 2000 de lire ...","Casa cu aburi","Albatros"))
    )
    override fun getBooks(): Set<Book> {
        return this.books
    }

    override fun addBook(book: Book) {
        this.books.add(book)
    }

    override fun findAllByAuthor(author: String): Set<Book> {
        return (this.books.filter { it.hasAuthor(author) }).toSet()
    }

    override fun findAllByTitle(title: String): Set<Book> {
        return (this.books.filter { it.hasTitle(title) }).toSet()
    }

    override fun findAllByPublisher(publisher: String): Set<Book> {
        return (this.books.filter { it.publishedBy(publisher) }).toSet()
    }
}

```

LibraryPrinterService

```

package com.sd.laborator.services

```

```

import com.sd.laborator.interfaces.LibraryPrinter
import com.sd.laborator.model.Book
import org.springframework.stereotype.Service

@Service
class LibraryPrinterService: LibraryPrinter {
    override fun printHTML(books: Set<Book>): String {
        var content: String = "<html><head><title>Libreria mea
HTML</title></head><body>"
        books.forEach {
            content +=
"<p><h3>${it.name}</h3><h4>${it.author}</h4><h5>${it.publisher}</h5>${it.content}</p><br/>"
        }
        content += "</body></html>"
        return content
    }

    override fun printJSON(books: Set<Book>): String {
        var content: String = "[\n"
        books.forEach {
            if (it != books.last())
                content += "    {\"Titlu\": \"${it.name}\",
\"Autor\": \"${it.author}\", \"Editura\": \"${it.publisher}\",
\"Text\": \"${it.content}\",\n"
            else
                content += "    {\"Titlu\": \"${it.name}\",
\"Autor\": \"${it.author}\", \"Editura\": \"${it.publisher}\",
\"Text\": \"${it.content}\",\n"
        }
        content += "]\n"
        return content
    }

    override fun printRaw(books: Set<Book>): String {
        var content: String = ""
        books.forEach {
            content +=
"${it.name}\n${it.author}\n${it.publisher}\n${it.content}\n\n"
        }
        return content
    }
}

```

LibraryApp

```
package com.sd.laborator
```

```

import org.springframework.boot.autoconfigure.SpringBootApplication
import org.springframework.boot.runApplication

@SpringBootApplication
open class LibraryApp

fun main(args: Array<String>) {
    runApplication<LibraryApp>(*args)
}

```

Application.properties

```
spring.datasource.url=jdbc:sqlite:library.db
```

Library_manager.py

```

import os
import sys
import requests
import qdarkstyle
from requests.exceptions import HTTPError
from PyQt5.QtWidgets import QWidget, QApplication, QFileDialog, QMessageBox
from PyQt5 import QtCore
from PyQt5.uic import loadUi

def debug_trace(ui=None):
    from pdb import set_trace
    QtCore.pyqtRemoveInputHook()
    set_trace()
    # QtCore.pyqtRestoreInputHook()

class LibraryApp(QWidget):
    ROOT_DIR = os.path.dirname(os.path.abspath(__file__))

    def __init__(self):
        super(LibraryApp, self).__init__()
        ui_path = os.path.join(self.ROOT_DIR, 'library_manager.ui')
        loadUi(ui_path, self)
        self.search_btn.clicked.connect(self.search)
        self.save_as_file_btn.clicked.connect(self.save_as_file)

    def search(self):
        search_string = self.search_bar.text()
        url = None
        if not search_string:

```

```

        if self.json_rb.isChecked():
            url = '/print?format=json'
        elif self.html_rb.isChecked():
            url = '/print?format=html'
        else:
            url = '/print?format=raw'
    else:
        if self.author_rb.isChecked():
            url = '/find?author={}'.format(search_string.replace(' ', '%20'))
        elif self.title_rb.isChecked():
            url = '/find?title={}'.format(search_string.replace(' ', '%20'))
        else:
            url = '/find?publisher={}'.format(search_string.replace(' ', '%20'))
    full_url = "http://localhost:8080" + url
    try:
        response = requests.get(full_url)
        self.result.setText(response.content.decode('utf-8'))
    except HTTPError as http_err:
        print('HTTP error occurred: {}'.format(http_err))
    except Exception as err:
        print('Other error occurred: {}'.format(err))

def save_as_file(self):
    options = QFileDialog.Options()
    options |= QFileDialog.DontUseNativeDialog
    file_path = str(
        QFileDialog.getSaveFileName(self,
                                    'Salvare fisier',
                                    options=options))

    if file_path:
        file_path = file_path.split('"')[1]
        if not file_path.endswith('.json') and not file_path.endswith(
            '.html') and not file_path.endswith('.txt'):
            if self.json_rb.isChecked():
                file_path += '.json'
            elif self.html_rb.isChecked():
                file_path += '.html'
            else:
                file_path += '.txt'
        try:
            with open(file_path, 'w') as fp:
                if file_path.endswith(".html"):
                    fp.write(self.result.toHtml())
                else:
                    fp.write(self.result.toPlainText())
        except Exception as e:
            print(e)

```

```
        QMessageBox.warning(self, 'Library Manager',
                             'Nu s-a putut salva fisierul')

if __name__ == '__main__':
    app = QApplication(sys.argv)

    stylesheet = qdarkstyle.load_stylesheet_pyqt5()
    app.setStyleSheet(stylesheet)

    window = LibraryApp()
    window.show()
    sys.exit(app.exec_())
```

PACHETE DE INSTALAT

```
PyQt5==5.12.3
pika==1.1.0
retry==0.9.2
QDarkStyle==2.8
requests
```

